

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



LAB MANUAL

Course No. : CSE 1202

Course Name : Data Structure Sessional

Semester : 1st Year Even Semester

Prepared By : Emrana Kabir Hashi, Assistant Professor, Department of Computer Science & Engineering (CSE), Rajshahi University of Engineering & Technology (RUET).



Data Structure Laboratory Guidelines

1. Before Coming to Lab

- **Read the experiment sheet/manual** carefully before the class.
- **Prepare pre-lab questions or algorithm steps** (if provided) and bring your lab notebook.
- **Review relevant theory** — understand the data structure concept involved (e.g., stack, queue, linked list, tree, graph).
- **Write or partially prepare the program** in advance (either pseudocode or C/C++/Python code as instructed).
- **Know the objective** of the experiment — what problem you are solving and what to demonstrate.

2. During Lab Session

A. Conduct

- Be **punctual** — attendance will be taken at the beginning of the session.
- Maintain **discipline and silence** in the lab; unnecessary conversation or browsing unrelated sites is not allowed.
- Use **lab computers carefully**; do not change system settings or delete any files.
- Respect **shared equipment and software** — use them responsibly and avoid tampering with any setup.

B. Work Process

- Each student or group must **write, compile, and execute** their own program.
- Follow the **step-by-step procedure** outlined in the experiment sheet.
- If errors occur:
 1. Check for **syntax or logical errors** in your code.
 2. Use **debugging statements or print outputs** to trace issues.
 3. Only then ask the instructor for guidance.
- Record **observations, outputs, and test results** neatly in your lab notebook.
- Work **independently** but cooperate when required during discussion or debugging.

3. After Completing Experiment

- **Demonstrate the working program** to the instructor for evaluation.
- **Explain your code** — algorithm, data structure used, and time complexity (if asked).
- Submit your lab notebook with the following sections:

- Aim & objective
- Algorithm / pseudocode
- Source code (well-commented)
- Sample input and output
- Explanation / trace table (if applicable)
- Conclusion and remarks
-  **Log off and tidy up** your workstation before leaving the lab.

4. Lab Report Writing

Each experiment report should include:

1. Experiment name & objective
2. Algorithm and brief theory
3. Flowchart or pseudocode (if applicable)
4. Program code (with comments)
5. Procedure followed
6. Input–output screenshots or results
7. Complexity analysis (time & space)
8. Precautions and limitations
9. Conclusion / summary of learning

5. General Rules & Precautions

-  **Do not copy code** from others — plagiarism leads to zero marks.
-  **Save your work frequently** to avoid data loss.
-  **Keep your folder organized** with clearly named files (e.g., Lab01_Stack.c).
-  **Avoid mobile usage** during the session unless for experiment reference.
-  **Do not modify or uninstall** lab software.
-  Maintain your **lab notebook** properly — it will be checked regularly.
-  Follow **faculty/lab assistant instructions** carefully.

6. Evaluation Criteria

Students will be graded on:

-  **Preparation** – pre-lab understanding and readiness.

-  **Implementation** – correctness and efficiency of program.
-  **Debugging skill** – ability to identify and fix logical/syntax errors.
-  **Understanding** – clarity in explaining algorithm and code.
-  **Reporting** – completeness, neatness, and originality of lab report.
-  **Discipline & teamwork** – behavior and cooperation in the lab.

Lab Curriculum

Table-1: CO-PO mapping for CSE 1202

CO	CO Statement	PO	Delivery Method	Assessments
CO1	Implement and manipulate linear data structures such as arrays, stacks, and queues to solve programming problems.	PO1	• Lectures and Tutorials	• Lab Experiments • Quiz • Lab Final
CO2	Design and develop solutions using data structures, considering performance and efficiency.	PO3	• Lectures and Tutorials	• Lab Experiments • Lab Final
CO3	Analyze and optimize algorithms for efficiency and performance using appropriate data structures	PO4	• Lectures and Tutorials	• Lab Experiments • Lab Final
CO4	Apply modern tools and techniques to implement and test data structures and algorithms.	PO5	• Lectures and Tutorials	• Lab Experiments • Lab Final

Table-2: PO statements

PO1	Engineering knowledge: Apply knowledge of mathematics, natural science, engineering fundamentals and an engineering specialization respectively to the solution of complex engineering problems.
PO2	Problem analysis: Identify, formulate, research literature and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.
PO3	Design/development of solutions: Design solutions for complex engineering problems and design systems, components or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations.
PO4	Investigation: Conduct investigations of complex problems using research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.
PO5	Modern tool usage: Create, select and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling, to complex engineering problems, with an understanding of the limitations.

PO6	The engineer and society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice and solutions to complex engineering problems.
PO7	Environment and sustainability: Understand and evaluate the sustainability and impact of professional engineering work in the solution of complex engineering problems in societal and environmental contexts.
PO8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
PO9	Individual work and teamwork: Function effectively as an individual, and as a member or leader in diverse teams and in multi-disciplinary settings.
PO10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	Project management and finance: Demonstrate knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

List of Experiments

Lab 1		Page Number
Experiment	Experiment on Linear Array: Implementation, Traversal, Insertion, and Deletion Operations	7 - 11
Lab 2		
Experiment	Experiment on Linear Search, Binary Search, Array Merging, and Array Splitting	12 - 18
Lab 3		
Experiment	Experiment on Stack: PUSH and POP Operations	19 - 23
Lab 4		
Experiment	Experiment on Stack: Transforming Infix Expression into Postfix Expression	24 - 28
Lab 5		
Experiment	Experiment on Queue: Insertion and Deletion Operation on Queue and Circular Queue	29 - 37
Lab 6		
Experiment	Experiment on Recursion: Factorial Function, Fibonacci Sequence, and Ackermann Function	38 - 42

Lab 7		
Experiment	Experiment on Linked List: Traversing, Searching, Insertion, and Deletion	43 - 50
Lab 8		
Experiment	Experiment on Linked List: Traversing, Searching, Insertion, and Deletion in a Doubly Linked List and Circular Linked List	51 - 58
Lab 9		
Experiment	Experiment on Sorting: Implementing and Analyzing Bubble Sort, Insertion Sort, and Selection Sort Algorithms	59 - 65
Lab 10		
Experiment	Experiment on Sorting: Implementing and Analyzing Merge Sort and Quick Sort Algorithms	66 - 71
Lab 11		
Experiment	Experiment on Tree: Traversing Binary Tree, Binary Search Tree, Insertion and Deletion on Binary Search Tree	72 - 79
Lab 12		
Experiment	Experiment on Insertion and Deletion on Heap, Heap Sort, Hash Table	80 - 86
Lab 13		
Exam	Lab test and Quiz	87 - 89

Lab 1

Experiment [CO1:PO1]

Name of the Experiment

Experiment on Linear Array: Implementation, Traversal, Insertion, and Deletion Operations

1. Introduction / Description

An **array** is one of the most fundamental **linear data structures** used to store a fixed number of elements of the same data type in contiguous memory locations. Each element can be accessed directly using its **index**, making array operations very efficient for searching and accessing.

In this experiment, we focus on performing the following basic operations on a **Linear Array**:

- **Implementation** – Declaring and initializing a linear array.
- **Traversal** – Accessing and displaying each element sequentially.
- **Insertion** – Adding a new element at a specified position by shifting elements to the right.
- **Deletion** – Removing an element from a specified position by shifting elements to the left.

These operations are the foundation for understanding memory management, data manipulation, and complexity analysis in algorithms.

2. Required Equipment / Software

Hardware:

- Computer with standard configuration
- Keyboard and mouse

Software:

- C Compiler (Code::Blocks / Turbo C / GCC / Dev-C++)
- Text editor (VS Code / Sublime Text / Notepad++)
- Operating System: Windows or Linux

3. Procedure

1. Setup Environment

- Open the C programming IDE (e.g., Code::Blocks or Turbo C++).

- Create a new project named **LinearArray_Operations**.
- Write or copy the provided program code into the editor.

2. Compile and Run the Program

- Compile the program and fix any syntax errors.
- Execute the program and observe the output for each operation.

3. Steps to Perform

- Input the total number of elements and initialize the array.
- Choose operations from the displayed menu:
 - Traversal – Display all elements.
 - Insertion – Add an element at a specified position.
 - Deletion – Remove an element from a specified position.
- Observe how the array changes after each operation.

4. Code / Pseudocode / Algorithm

Algorithm Steps

A. Traversal

1. Start
2. Read all elements of the array
3. Print each element using a loop
4. Stop

B. Insertion

1. Start
2. Input position and new element
3. Shift elements right from the end to the insertion position
4. Insert the new element
5. Increase array size by one
6. Display updated array
7. Stop

C. Deletion

1. Start

2. Input position of element to delete
3. Shift elements left from next position onward
4. Decrease array size by one
5. Display updated array
6. Stop

C Program Implementation

```
#include <stdio.h>

int main() {
    int arr[100], n, i, pos, val, choice;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for(i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    while(1) {
        printf("\n--- Linear Array Operations ---\n");
        printf("1. Traversal\n2. Insertion\n3. Deletion\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1:
                printf("Array elements: ");
                for(i = 0; i < n; i++)
                    printf("%d ", arr[i]);
                printf("\n");
                break;
            case 2:
                printf("Enter position (0 to %d): ", n);
                scanf("%d", &pos);
```

```

printf("Enter value to insert: ");
scanf("%d", &val);
if(pos < 0 || pos > n)
    printf("Invalid position!\n");
else {
    for(i = n; i > pos; i--)
        arr[i] = arr[i - 1];
    arr[pos] = val;
    n++;
    printf("After insertion: ");
    for(i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
break;
case 3:
    printf("Enter position to delete (0 to %d): ", n - 1);
    scanf("%d", &pos);
    if(pos < 0 || pos >= n)
        printf("Invalid position!\n");
    else {
        for(i = pos; i < n - 1; i++)
            arr[i] = arr[i + 1];
        n--;
        printf("After deletion: ");
        for(i = 0; i < n; i++)
            printf("%d ", arr[i]);
        printf("\n");
    }
}

```

```

        break;

    case 4:

        return 0;

    default:

        printf("Invalid choice! Try again.\n");

    }

}

return 0;

}

```

5. Precautions

- Ensure that the **position index** for insertion or deletion is valid (within 0 to $n-1$).
- Avoid **array overflow** — do not insert beyond the declared size.
- Prevent **underflow** — do not delete when the array is empty.
- Use appropriate **data types** for the array depending on the data range.
- Save and test your code regularly to avoid losing progress.

6. Conclusion / Discussion

This experiment demonstrates how to implement and manipulate a **linear array** through **traversal, insertion, and deletion operations**.

Students learn to:

- Represent data in contiguous memory using arrays.
- Perform dynamic updates on an array by shifting elements.
- Understand the **time complexity** of operations:
 - Traversal: $O(n)$
 - Insertion: $O(n)$
 - Deletion: $O(n)$

By completing this experiment, students gain practical understanding of array manipulation and how these basic concepts form the foundation for advanced data structures such as stacks, queues, and linked lists.

Lab 2

Experiment [CO2:PO3]

Name of the Experiment

Experiment on Linear Search, Binary Search, Array Merging, and Array Splitting

1. Introduction / Description

Searching and combining operations are fundamental in data structures and algorithms. Searching refers to the process of finding the position or existence of a particular element in a data collection, while merging and splitting deal with organizing and managing multiple arrays efficiently.

In this experiment, we focus on performing the following operations on arrays:

- **Linear Search** – Sequentially searching an element from the beginning to the end of the array.
- **Binary Search** – Efficiently finding an element in a **sorted** array by repeatedly dividing the search interval in half.
- **Array Merging** – Combining two arrays into a single larger array.
- **Array Splitting** – Dividing one array into two or more smaller arrays based on user-defined conditions or indices.

These operations are essential for understanding data access, organization, and optimization techniques in algorithm design.

2. Required Equipment / Software

Hardware:

- Computer with standard configuration
- Keyboard and mouse

Software:

- C Compiler (Code::Blocks / Turbo C / GCC / Dev-C++)
- Text editor (VS Code / Sublime Text / Notepad++)
- Operating System: Windows or Linux

3. Procedure

Step 1: Setup Environment

- Open the C programming IDE (e.g., Code::Blocks or Turbo C++).
- Create a new project named **Array_Search_Merge_Split**.
- Write or copy the provided program code into the editor.

Step 2: Compile and Run

- Compile the program and fix any syntax errors.
- Execute the program and test each operation.

Step 3: Steps to Perform

- Input the size and elements of one or two arrays as required.
- Choose from the operation menu:
 - Linear Search – Search for an element sequentially.
 - Binary Search – Search efficiently in a sorted array.
 - Merge Arrays – Combine two arrays into one.
 - Split Array – Divide an array into two subarrays.
- Observe the output after each operation.

4. Code / Pseudocode / Algorithm

A. Linear Search Algorithm

1. Start
2. Input array elements and target element
3. Compare target with each element sequentially
4. If found, display the position
5. If not found, display “Element not found”
6. Stop

B. Binary Search Algorithm

1. Start
2. Input sorted array elements and target element
3. Initialize low = 0, high = n - 1
4. Repeat until low <= high:
 - mid = (low + high) / 2
 - If arr[mid] == target, return position
 - If arr[mid] < target, set low = mid + 1
 - Else, set high = mid - 1
5. Display “Element not found” if not present
6. Stop

C. Array Merging Algorithm

1. Start
2. Input elements of Array A and Array B
3. Create a new array C

4. Copy all elements of A, then all elements of B into C
5. Display the merged array
6. Stop

D. Array Splitting Algorithm

1. Start
2. Input elements of the main array
3. Input position p where to split
4. Copy elements before p into Array A1
5. Copy remaining elements into Array A2
6. Display both subarrays
7. Stop

C Program Implementation

```
#include <stdio.h>
```

```
int main() {
```

```
    int arr1[100], arr2[100], merged[200];
```

```
    int n1, n2, n, i, j, pos, val, choice, key, low, high, mid, found;
```

```
    while (1) {
```

```
        printf("\n--- Array Operations ---\n");
```

```
        printf("1. Linear Search\n2. Binary Search\n3. Merge Arrays\n4. Split Array\n5. Exit\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch (choice) {
```

```
            case 1: // Linear Search
```

```
                printf("Enter number of elements: ");
```

```
                scanf("%d", &n);
```

```

printf("Enter elements: ");

for (i = 0; i < n; i++) scanf("%d", &arr1[i]);

printf("Enter element to search: ");

scanf("%d", &key);

found = 0;

for (i = 0; i < n; i++) {

    if (arr1[i] == key) {

        printf("Element found at position %d\n", i);

        found = 1;

        break;

    }

}

if (!found) printf("Element not found!\n");

break;

```

case 2: // Binary Search

```

printf("Enter number of elements (sorted): ");

scanf("%d", &n);

printf("Enter sorted elements: ");

for (i = 0; i < n; i++) scanf("%d", &arr1[i]);

printf("Enter element to search: ");

scanf("%d", &key);

low = 0; high = n - 1; found = 0;

while (low <= high) {

    mid = (low + high) / 2;

```

```

if (arr1[mid] == key) {
    printf("Element found at position %d\n", mid);
    found = 1; break;
} else if (arr1[mid] < key)
    low = mid + 1;
else
    high = mid - 1;
}
if (!found) printf("Element not found!\n");
break;

```

case 3: // Merge Arrays

```

printf("Enter number of elements in first array: ");
scanf("%d", &n1);
printf("Enter elements of first array: ");
for (i = 0; i < n1; i++) scanf("%d", &arr1[i]);
printf("Enter number of elements in second array: ");
scanf("%d", &n2);
printf("Enter elements of second array: ");
for (i = 0; i < n2; i++) scanf("%d", &arr2[i]);
for (i = 0; i < n1; i++) merged[i] = arr1[i];
for (j = 0; j < n2; j++) merged[n1 + j] = arr2[j];
printf("Merged Array: ");
for (i = 0; i < n1 + n2; i++) printf("%d ", merged[i]);
printf("\n");

```

```
break;
```

```
case 4: // Split Array
```

```
printf("Enter number of elements: ");
```

```
scanf("%d", &n);
```

```
printf("Enter elements: ");
```

```
for (i = 0; i < n; i++) scanf("%d", &arr1[i]);
```

```
printf("Enter position to split (1 to %d): ", n - 1);
```

```
scanf("%d", &pos);
```

```
printf("First part: ");
```

```
for (i = 0; i < pos; i++) printf("%d ", arr1[i]);
```

```
printf("\nSecond part: ");
```

```
for (i = pos; i < n; i++) printf("%d ", arr1[i]);
```

```
printf("\n");
```

```
break;
```

```
case 5:
```

```
return 0;
```

```
default:
```

```
printf("Invalid choice! Try again.\n");
```

```
}
```

```
}
```

```
}
```

5. Precautions

- Ensure arrays are declared with sufficient size before merging.
- Perform binary search **only** on sorted arrays.
- Validate split positions (must be between 1 and $n-1$).
- Avoid entering invalid indices or exceeding array limits.
- Save your code frequently to prevent data loss.

6. Conclusion / Discussion

This experiment demonstrates the use of different array-based operations for searching and organizing data.

Students learn to:

- Implement **Linear and Binary Search** and compare their efficiency.
- Perform **Array Merging** and **Splitting** operations effectively.
- Analyze the **time complexity** of each operation:
 - Linear Search: $O(n)$
 - Binary Search: $O(\log n)$
 - Merge: $O(n_1 + n_2)$
 - Split: $O(n)$

By completing this experiment, students gain practical understanding of array-based algorithms that are foundational for advanced topics like sorting, searching optimization, and data structure manipulation.

Lab 3

Experiment [CO1:PO1]

Name of the Experiment

Experiment on Stack: PUSH and POP Operations

1. Introduction / Description

A **stack** is a fundamental **linear data structure** that follows the **LIFO (Last In, First Out)** principle — meaning the last element inserted is the first one to be removed. It is widely used in **expression evaluation, recursion management, undo mechanisms, and compiler design**.

In this experiment, we focus on performing the following fundamental stack operations:

- **PUSH Operation** – Inserting an element at the top of the stack.
- **POP Operation** – Removing the element from the top of the stack.

These operations help in understanding how data is stored and retrieved in restricted data structures and form the basis for more advanced concepts such as function calls, backtracking, and parsing.

2. Required Equipment / Software

Hardware:

- Computer with standard configuration
- Keyboard and mouse

Software:

- C Compiler (Code::Blocks / Turbo C / GCC / Dev-C++)
- Text editor (VS Code / Sublime Text / Notepad++)
- Operating System: Windows or Linux

3. Procedure

Step 1: Setup Environment

- Open the C programming IDE (e.g., Code::Blocks or Turbo C++).
- Create a new project named **Stack_Push_Pop**.
- Write or copy the provided program code into the editor.

Step 2: Compile and Run

- Compile the code and fix any syntax errors.
- Run the program and choose operations from the displayed menu.

Step 3: Steps to Perform

- Input the size of the stack.
- Select from the operation menu:
 - **PUSH** – Insert an element into the stack.
 - **POP** – Remove an element from the stack.
 - **Display** – Show all current elements in the stack.
 - **Exit** – Terminate the program.
- Observe how the stack changes after each operation.

4. Code / Pseudocode / Algorithm

A. PUSH Operation Algorithm

1. Start
2. Check if $\text{top} == \text{max} - 1$ (Stack Overflow)
3. If true, display “Stack Overflow” and stop
4. Else, input the element to push
5. Increment top by 1
6. Insert the element at $\text{stack}[\text{top}]$
7. Display updated stack
8. Stop

B. POP Operation Algorithm

1. Start
2. Check if $\text{top} == -1$ (Stack Underflow)
3. If true, display “Stack Underflow” and stop
4. Else, remove element from $\text{stack}[\text{top}]$
5. Decrement top by 1
6. Display the popped element and updated stack
7. Stop

C. Display Operation Algorithm

1. Start
2. If $\text{top} == -1$, display “Stack is Empty”
3. Else, display all elements from top to 0
4. Stop

5. C Program Implementation

```
#include <stdio.h>

#define MAX 100

int stack[MAX];

int top = -1;

void push() {
    int val;

    if (top == MAX - 1) {
        printf("Stack Overflow! Cannot push element.\n");
    } else {
        printf("Enter value to push: ");

        scanf("%d", &val);

        top++;

        stack[top] = val;

        printf("Element %d pushed successfully.\n", val);
    }
}

void pop() {
    if (top == -1) {
        printf("Stack Underflow! No elements to pop.\n");
    } else {
        printf("Element %d popped from stack.\n", stack[top]);

        top--;
    }
}
```

```

}

void display() {
    int i;

    if (top == -1) {
        printf("Stack is empty.\n");
    } else {
        printf("Stack elements (Top to Bottom): ");

        for (i = top; i >= 0; i--)
            printf("%d ", stack[i]);

        printf("\n");
    }
}

int main() {
    int choice;

    while (1) {
        printf("\n--- Stack Operations ---\n");

        printf("1. PUSH\n2. POP\n3. DISPLAY\n4. EXIT\n");

        printf("Enter your choice: ");

        scanf("%d", &choice);

        switch (choice) {
            case 1:
                push();

                break;

            case 2:
                pop();

```

```

        break;

    case 3:

        display();

        break;

    case 4:

        return 0;

    default:

        printf("Invalid choice! Try again.\n");

    }

}

}

```

6. Precautions

- Ensure the stack does not exceed its declared size (**avoid overflow**).
- Do not perform **POP** on an empty stack (**avoid underflow**).
- Validate user inputs to maintain correct data types.
- Save and test your code frequently.
- Always initialize top = -1 before performing stack operations.

7. Conclusion / Discussion

This experiment demonstrates the basic **PUSH** and **POP** operations in a stack — a vital linear data structure used for managing ordered data with restricted access.

Students learn to:

- Implement stack operations using an **array-based approach**.
- Understand **overflow** and **underflow** conditions.
- Analyze the time complexity of operations:
 - PUSH: O(1)
 - POP: O(1)
 - Display: O(n)

By completing this experiment, students gain practical insight into how stacks work, setting the foundation for advanced applications such as **recursion, expression evaluation, and function call management**.

Lab 4

Experiment [CO2:PO3]

Name of the Experiment

Experiment on Stack: Transforming Infix Expression into Postfix Expression

1. Introduction / Description

Stacks are one of the most important linear data structures used in expression evaluation and syntax conversion in computer science.

An **infix expression** is an arithmetic expression where the operator is written **between** the operands (e.g., $A + B$). However, for efficient computation, infix expressions are often converted into **postfix form** (also known as **Reverse Polish Notation**), where the operator follows the operands (e.g., $A B +$).

The stack data structure plays a crucial role in this conversion process. During the transformation, operators and parentheses are temporarily stored in a stack to maintain correct precedence and associativity.

Objective:

To implement a program that converts an infix expression into its equivalent postfix expression using a stack.

Key Concepts:

- Stack operations (PUSH and POP)
- Operator precedence and associativity
- Infix to Postfix conversion rules

2. Required Equipment / Software

Hardware:

- Computer with standard configuration
- Keyboard and mouse

Software:

- C Compiler (Code::Blocks / Turbo C / GCC / Dev-C++)
- Text editor (VS Code / Sublime Text / Notepad++)
- Operating System: Windows or Linux

3. Procedure

Step 1: Setup Environment

- Open your preferred C programming IDE.
- Create a new project named Infix_To_Postfix_Conversion.
- Write or copy the provided source code into the editor.

Step 2: Compile and Run

- Compile the program and correct any syntax errors.
- Execute the program and enter a valid infix expression as input.

Step 3: Steps to Perform

1. Enter a valid infix expression (e.g., (A+B)*(C-D)).
2. The program will use a stack to temporarily store operators.
3. Operators are popped from the stack according to precedence rules.
4. The resulting postfix expression is displayed.

4. Algorithm / Pseudocode

Algorithm: Infix to Postfix Conversion

1. **Start**
2. Input an infix expression as a string.
3. Initialize an empty stack for operators.
4. For each symbol in the infix expression:
 - a. If the symbol is an **operand**, add it to the postfix expression.
 - b. If the symbol is an '(', push it onto the stack.
 - c. If the symbol is an ')', pop operators until a '(' is encountered.
 - d. If the symbol is an **operator** (+, -, *, /, ^):
 - While the stack is not empty and the precedence of the top operator $\geq$ current operator, pop and add it to postfix.
 - Push the current operator onto the stack.
5. After scanning the entire infix expression, pop all remaining operators from the stack to postfix.
6. **Display** the final postfix expression.
7. **Stop**

5. C Program Implementation

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#include <string.h>

#define MAX 100

char stack[MAX];

int top = -1;

void push(char c) {
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
    } else {
        stack[++top] = c;
    }
}

char pop() {
    if (top == -1) {
        return '\0';
    } else {
        return stack[top--];
    }
}

int precedence(char op) {
    switch (op) {
        case '^': return 3;
        case '*':
        case '/': return 2;
        case '+':
        case '-': return 1;
    }
}
```

```

        default: return 0;

    }

}

void infixToPostfix(char infix[]) {

    char postfix[MAX], ch, x;

    int i = 0, k = 0;

    while ((ch = infix[i++]) != '\0') {

        if (isalnum(ch)) {

            postfix[k++] = ch;

        } else if (ch == '(') {

            push(ch);

        } else if (ch == ')') {

            while ((x = pop()) != '(')

                postfix[k++] = x;

        } else {

            while (precedence(stack[top]) >= precedence(ch))

                postfix[k++] = pop();

            push(ch);

        }

    }

    while (top != -1)

        postfix[k++] = pop();

    postfix[k] = '\0';

    printf("Postfix Expression: %s\n", postfix);

}

```

```

int main() {

    char infix[MAX];

    printf("Enter an infix expression: ");

    gets(infix);

    infixToPostfix(infix);

    return 0;

}

```

6. Precautions

- Ensure that parentheses in the infix expression are balanced.
- Avoid invalid operators or missing operands.
- Clear the stack before each new conversion.
- Use only alphanumeric operands (A–Z, 0–9).
- Check for stack overflow or underflow conditions.

7. Conclusion / Discussion

In this experiment, we successfully implemented the conversion of infix expressions to postfix form using a **stack**. The experiment highlights how the **stack** helps manage operator precedence and associativity efficiently.

Students learned to:

- Apply stack operations (PUSH/POP) for expression manipulation.
- Implement and understand the logic of operator precedence and associativity.
- Compare infix and postfix notations in terms of evaluation simplicity.

Time Complexity:

- Scanning each element once $\rightarrow O(n)$
 - Each operator pushed and popped once $\rightarrow O(n)$
- Overall complexity: $O(n)$**

By completing this experiment, students gain a solid understanding of stack applications in **compiler design**, **expression parsing**, and **evaluation algorithms**.

Lab 5

Experiment [CO1:PO1]

Name of the Experiment

Experiment on Queue: Insertion and Deletion Operation on Queue and Circular Queue

1. Introduction / Description

A **queue** is a linear data structure that follows the **FIFO (First In, First Out)** principle, where the element inserted first is removed first. Queues are widely used in **process scheduling, resource management, and buffer handling**.

There are two types of queue implementations in this experiment:

1. **Linear Queue** – A simple queue where insertion is performed at the rear and deletion from the front.
2. **Circular Queue** – An improved queue that reuses vacant positions in a linear array using modular arithmetic to avoid wasted space.

This experiment demonstrates the **insertion (enqueue)** and **deletion (dequeue)** operations on both linear and circular queues. Students will learn to handle overflow and underflow conditions effectively.

2. Required Equipment / Software

Hardware:

- Computer with standard configuration
- Keyboard and mouse

Software:

- C Compiler (Code::Blocks / Turbo C / GCC / Dev-C++)
- Text editor (VS Code / Sublime Text / Notepad++)
- Operating System: Windows or Linux

3. Procedure

Step 1: Setup Environment

- Open the C programming IDE (e.g., Code::Blocks or Turbo C++).
- Create a new project named Queue_Operations.
- Write or copy the provided program code into the editor.

Step 2: Compile and Run

- Compile the program and fix any syntax errors.
- Execute the program and test the queue operations.

Step 3: Steps to Perform

1. Input the size of the queue.
2. Choose the queue type: Linear or Circular.
3. Perform operations:
 - **Insertion (Enqueue)** – Add an element to the queue.
 - **Deletion (Dequeue)** – Remove an element from the queue.
 - **Display** – Show all elements in the queue.
 - **Exit** – Terminate the program.
4. Observe the queue behavior and check for **overflow** and **underflow** conditions.

4. Algorithm / Pseudocode

A. Linear Queue Operations

Enqueue (Insertion):

1. Start
2. Check if $\text{rear} == \text{max} - 1 \rightarrow \text{Overflow}$
3. If no, increment rear and insert the element at $\text{queue}[\text{rear}]$
4. Stop

Dequeue (Deletion):

1. Start
2. Check if $\text{front} > \text{rear} \rightarrow \text{Underflow}$
3. If no, remove element from $\text{queue}[\text{front}]$ and increment front
4. Stop

B. Circular Queue Operations

Enqueue (Insertion):

1. Start

2. Check if $(\text{rear} + 1) \% \text{max} == \text{front} \rightarrow \text{Overflow}$
3. If no, $\text{rear} = (\text{rear} + 1) \% \text{max}$ and insert element at $\text{queue}[\text{rear}]$
4. Stop

Deque (Deletion):

1. Start
2. Check if $\text{front} == -1 \rightarrow \text{Underflow}$
3. If no, remove element at $\text{queue}[\text{front}]$
4. If $\text{front} == \text{rear}$, reset $\text{front} = \text{rear} = -1$
5. Else, $\text{front} = (\text{front} + 1) \% \text{max}$
6. Stop

5. C Program Implementation

```
#include <stdio.h>
```

```
#define MAX 100
```

```
int queue[MAX];
```

```
int front = -1, rear = -1;
```

```
// Linear Queue Functions
```

```
void enqueueLinear(int val, int n) {
```

```
    if (rear == n - 1) {
```

```
        printf("Queue Overflow!\n");
```

```
    } else {
```

```
        if (front == -1) front = 0;
```

```
        rear++;
```

```
        queue[rear] = val;
```

```
        printf("Inserted %d into Linear Queue.\n", val);
```

```
    }
```

```
}
```

```
void dequeueLinear() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow!\n");
    } else {
        printf("Deleted %d from Linear Queue.\n", queue[front]);
        front++;
    }
}
```

```
void displayLinear() {
    if (front == -1 || front > rear) {
        printf("Queue is empty.\n");
    } else {
        printf("Linear Queue elements: ");
        for (int i = front; i <= rear; i++)
            printf("%d ", queue[i]);
        printf("\n");
    }
}
```

```
// Circular Queue Functions
```

```
int cfront = -1, crear = -1;
```

```

void enqueueCircular(int val, int n) {
    if ((crear + 1) % n == cfront) {
        printf("Circular Queue Overflow!\n");
    } else {
        if (cfront == -1) cfront = 0;
        crear = (crear + 1) % n;
        queue[crear] = val;
        printf("Inserted %d into Circular Queue.\n", val);
    }
}

```

```

void dequeueCircular(int n) {
    if (cfront == -1) {
        printf("Circular Queue Underflow!\n");
    } else {
        printf("Deleted %d from Circular Queue.\n", queue[cfront]);
        if (cfront == crear) {
            cfront = crear = -1;
        } else {
            cfront = (cfront + 1) % n;
        }
    }
}

```

```

void displayCircular(int n) {

```

```

if (cfront == -1) {

    printf("Circular Queue is empty.\n");

} else {

    printf("Circular Queue elements: ");

    int i = cfront;

    while (1) {

        printf("%d ", queue[i]);

        if (i == crear) break;

        i = (i + 1) % n;

    }

    printf("\n");

}

}

int main() {

    int choice, val, n, type;

    printf("Enter size of queue: ");

    scanf("%d", &n);

    while (1) {

        printf("\n--- Queue Operations ---\n");

        printf("1. Linear Queue\n2. Circular Queue\n3. Exit\n");

        printf("Choose queue type: ");

        scanf("%d", &type);

```

```
if (type == 3) break;

if (type == 1) {
    printf("\n--- Linear Queue ---\n");
    printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Back\n");
    printf("Enter choice: ");
    scanf("%d", &choice);
    switch (choice) {
        case 1:
            printf("Enter value to insert: ");
            scanf("%d", &val);
            enqueueLinear(val, n);
            break;
        case 2:
            dequeueLinear();
            break;
        case 3:
            displayLinear();
            break;
        case 4:
            break;
        default:
            printf("Invalid choice!\n");
    }
} else if (type == 2) {
```

```

printf("\n--- Circular Queue ---\n");

printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Back\n");

printf("Enter choice: ");

scanf("%d", &choice);

switch (choice) {

    case 1:

        printf("Enter value to insert: ");

        scanf("%d", &val);

        enqueueCircular(val, n);

        break;

    case 2:

        dequeueCircular(n);

        break;

    case 3:

        displayCircular(n);

        break;

    case 4:

        break;

    default:

        printf("Invalid choice!\n");

}

} else {

    printf("Invalid queue type!\n");

}

}

```

```
    return 0;  
}
```

6. Precautions

- Ensure queue size is sufficient for the operations.
- Avoid inserting when the queue is full (**Overflow**).
- Avoid deleting from an empty queue (**Underflow**).
- Reset front and rear properly for circular queue after last deletion.
- Use consistent array indices and modular arithmetic for circular queue.

7. Conclusion / Discussion

This experiment demonstrates **insertion and deletion operations on linear and circular queues**, emphasizing **FIFO behavior**.

Students learned to:

- Implement **linear and circular queues** using arrays.
- Handle **overflow and underflow** conditions effectively.
- Compare **linear vs circular queue efficiency** in utilizing array space.
- Analyze **time complexity** of operations:
 - Enqueue: $O(1)$
 - Dequeue: $O(1)$
 - Display: $O(n)$

By completing this experiment, students gain practical insight into queue operations and understand the significance of **circular queues** in optimizing memory usage.

Lab 6

Experiment [CO2:PO3]

Name of the Experiment

Experiment on Recursion: Factorial Function, Fibonacci Sequence, and Ackermann Function

1. Introduction / Description

Recursion is a programming technique in which a function calls itself directly or indirectly to solve a problem. Recursive methods are widely used in algorithm design for tasks such as **mathematical computations, data structure traversal, and problem decomposition.**

In this experiment, students will implement recursive functions to compute:

1. **Factorial Function** – Calculates the factorial of a number $n! = n \times (n-1) \times \dots \times 1$.
2. **Fibonacci Sequence** – Generates the sequence where each term is the sum of the two preceding terms ($F(n) = F(n-1) + F(n-2)$).
3. **Ackermann Function** – Demonstrates a highly recursive function with deep recursion and non-linear growth:

$$A(m, n) = n+1 \text{ if } m=0,$$

$$A(m, n) = A(m-1, 1) \text{ if } m>0 \text{ and } n=0,$$

$$A(m, n) = A(m-1, A(m, n-1)) \text{ if } m>0 \text{ and } n>0.$$

These exercises illustrate **recursive problem-solving, function calls, and stack usage** in computer memory.

2. Required Equipment / Software

Hardware:

- Computer with standard configuration
- Keyboard and mouse

Software:

- C Compiler (Code::Blocks / Turbo C / GCC / Dev-C++)
- Text editor (VS Code / Sublime Text / Notepad++)
- Operating System: Windows or Linux

3. Procedure

Step 1: Setup Environment

- Open the C programming IDE (e.g., Code::Blocks or Turbo C++).
- Create a new project named Recursion_Functions.
- Write or copy the provided program code into the editor.

Step 2: Compile and Run

- Compile the program and fix any syntax errors.
- Execute the program and test each recursive function.

Step 3: Steps to Perform

1. Choose the recursive function to execute: Factorial, Fibonacci, or Ackermann.
2. Input the required numbers (e.g., n for factorial, n for Fibonacci, m and n for Ackermann).
3. Observe the output and verify correctness.
4. Repeat with different inputs to test recursion depth and behavior.

4. Algorithm / Pseudocode

A. Factorial Function

1. Start
2. If $n == 0$ or $n == 1$, return 1
3. Else return $n * \text{factorial}(n - 1)$
4. Stop

B. Fibonacci Sequence

1. Start
2. If $n == 0$, return 0; if $n == 1$, return 1
3. Else return $\text{fibonacci}(n-1) + \text{fibonacci}(n-2)$
4. Stop

C. Ackermann Function

1. Start
2. If $m == 0$, return $n + 1$
3. Else if $n == 0$, return $\text{ackermann}(m-1, 1)$
4. Else return $\text{ackermann}(m-1, \text{ackermann}(m, n-1))$
5. Stop

5. C Program Implementation

```
#include <stdio.h>
```

```
// Factorial Function
```

```
int factorial(int n) {
```

```

if (n == 0 || n == 1)
    return 1;
else
    return n * factorial(n - 1);
}

```

// Fibonacci Function

```

int fibonacci(int n) {
    if (n == 0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}

```

// Ackermann Function

```

int ackermann(int m, int n) {
    if (m == 0)
        return n + 1;
    else if (n == 0)
        return ackermann(m - 1, 1);
    else
        return ackermann(m - 1, ackermann(m, n - 1));
}

```

```

int main() {

    int choice, n, m;

    while (1) {

        printf("\n--- Recursion Operations ---\n");

        printf("1. Factorial\n2. Fibonacci\n3. Ackermann\n4. Exit\n");

        printf("Enter your choice: ");

        scanf("%d", &choice);

        switch (choice) {

            case 1:

                printf("Enter n for factorial: ");

                scanf("%d", &n);

                printf("Factorial(%d) = %d\n", n, factorial(n));

                break;

            case 2:

                printf("Enter n for Fibonacci sequence: ");

                scanf("%d", &n);

                printf("Fibonacci(%d) = %d\n", n, fibonacci(n));

                break;

            case 3:

                printf("Enter m and n for Ackermann function: ");

                scanf("%d %d", &m, &n);

                printf("Ackermann(%d, %d) = %d\n", m, n, ackermann(m, n));

```

```

        break;

    case 4:

        return 0;

    default:

        printf("Invalid choice! Try again.\n");

    }

}

return 0;

}

```

6. Precautions

- Ensure input values for **Ackermann function** are **small**, as recursion depth grows extremely fast.
- Avoid entering large n for Fibonacci to prevent stack overflow.
- Check for negative inputs and handle them appropriately.
- Save the code frequently to prevent loss during testing.
- Understand recursion call stack to debug errors effectively.

7. Conclusion / Discussion

This experiment demonstrates the use of recursion in solving **mathematical problems**.

Students learned to:

- Implement **factorial** using recursion.
- Generate **Fibonacci sequence** recursively.
- Implement **Ackermann function** to understand deep recursion and stack behavior.
- Analyze **time complexity** and recursive function efficiency:
 - Factorial: $O(n)$
 - Fibonacci (naive recursive): $O(2^n)$
 - Ackermann: Extremely high non-primitive recursive growth

By completing this experiment, students gain practical understanding of recursion, stack usage, and the efficiency implications of recursive algorithms.

Lab 7

Experiment [CO1:PO1]

Name of the Experiment

Experiment on Linked List: Traversing, Searching, Insertion, and Deletion

1. Introduction / Description

A **linked list** is a dynamic linear data structure consisting of nodes, where each node contains data and a pointer to the next node. Unlike arrays, linked lists do not require contiguous memory allocation and allow efficient insertion and deletion operations.

In this experiment, students will perform the following operations on a singly linked list:

- **Traversal** – Access and display each element sequentially.
- **Searching** – Find the presence and position of a specific element.
- **Insertion** – Add a new node at the beginning, end, or a specific position.
- **Deletion** – Remove a node from the beginning, end, or a specific position.

These operations are fundamental for understanding **dynamic memory management, pointer manipulation, and data structure optimization.**

2. Required Equipment / Software

Hardware:

- Computer with standard configuration
- Keyboard and mouse

Software:

- C Compiler (Code::Blocks / Turbo C / GCC / Dev-C++)
- Text editor (VS Code / Sublime Text / Notepad++)
- Operating System: Windows or Linux

3. Procedure

Step 1: Setup Environment

- Open the C programming IDE (e.g., Code::Blocks or Turbo C++).
- Create a new project named LinkedList_Operations.
- Write or copy the provided program code into the editor.

Step 2: Compile and Run

- Compile the program and fix any syntax errors.
- Execute the program and test each operation interactively.

Step 3: Steps to Perform

1. Choose operations from the displayed menu: Traversal, Searching, Insertion, Deletion.
2. Input the values and positions as required.
3. Observe the output after each operation.
4. Repeat to understand how the linked list structure changes dynamically.

4. Algorithm / Pseudocode

A. Traversal

1. Start from the head node.
2. While the current node is not NULL:
 - Print the data of the current node.
 - Move to the next node.
3. Stop.

B. Searching

1. Start from the head node.
2. While the current node is not NULL:
 - Compare the node's data with the target.
 - If found, display position and exit.
 - Else, move to the next node.
3. If not found, display "Element not found".

C. Insertion

1. Create a new node with input data.
2. Depending on position:
 - Beginning: Point new node to head, update head.
 - End: Traverse to last node, point last node's next to new node.
 - Specific position: Traverse to position-1, insert node.

D. Deletion

1. Depending on position:
 - Beginning: Update head to next node, free first node.

- End: Traverse to second last node, free last node.
- Specific position: Traverse to position-1, adjust pointers, free node.

5. C Program Implementation

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node* next;  
};
```

```
struct Node* head = NULL;
```

```
// Traversal
```

```
void traverse() {  
    struct Node* ptr = head;  
    if (ptr == NULL) {  
        printf("List is empty.\n");  
        return;  
    }  
    printf("Linked List: ");  
    while (ptr != NULL) {  
        printf("%d ", ptr->data);  
        ptr = ptr->next;  
    }
```

```

    }

    printf("\n");
}

```

// Searching

```

void search(int key) {
    struct Node* ptr = head;

    int pos = 0, found = 0;

    while (ptr != NULL) {
        if (ptr->data == key) {
            printf("Element %d found at position %d\n", key, pos);

            found = 1;

            break;
        }

        ptr = ptr->next;

        pos++;
    }

    if (!found) printf("Element %d not found.\n", key);
}

```

// Insertion

```

void insert(int val, int pos) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = val;

    newNode->next = NULL;
}

```

```

if (pos == 0) {
    newNode->next = head;
    head = newNode;
    return;
}

```

```

struct Node* ptr = head;

for (int i = 0; i < pos - 1 && ptr != NULL; i++)
    ptr = ptr->next;

```

```

if (ptr == NULL) {
    printf("Position out of range.\n");
    free(newNode);
    return;
}

```

```

newNode->next = ptr->next;
ptr->next = newNode;
}

```

// Deletion

```

void deleteNode(int pos) {
    if (head == NULL) {
        printf("List is empty.\n");
    }
}

```

```

    return;
}

```

```

struct Node* temp;

if (pos == 0) {
    temp = head;
    head = head->next;
    free(temp);
    return;
}

```

```

struct Node* ptr = head;

for (int i = 0; i < pos - 1 && ptr != NULL; i++)
    ptr = ptr->next;

if (ptr == NULL || ptr->next == NULL) {
    printf("Position out of range.\n");
    return;
}

```

```

temp = ptr->next;
ptr->next = temp->next;
free(temp);
}

```

```
int main() {  
  
    int choice, val, pos, key;  
  
    while (1) {  
  
        printf("\n--- Linked List Operations ---\n");  
  
        printf("1. Traversal\n2. Search\n3. Insertion\n4. Deletion\n5. Exit\n");  
  
        printf("Enter your choice: ");  
  
        scanf("%d", &choice);  
  
  
        switch(choice) {  
  
            case 1: traverse(); break;  
  
            case 2:  
  
                printf("Enter element to search: ");  
  
                scanf("%d", &key);  
  
                search(key);  
  
                break;  
  
            case 3:  
  
                printf("Enter value to insert: ");  
  
                scanf("%d", &val);  
  
                printf("Enter position to insert: ");  
  
                scanf("%d", &pos);  
  
                insert(val, pos);  
  
                break;  
  
            case 4:  
  
                printf("Enter position to delete: ");  
  
                scanf("%d", &pos);
```

```

        deleteNode(pos);

        break;

    case 5: return 0;

    default: printf("Invalid choice! Try again.\n");

}

}

}

```

6. Precautions

- Always check for **NULL pointers** before accessing nodes.
- Validate positions for insertion and deletion.
- Free memory for deleted nodes to avoid **memory leaks**.
- Avoid negative indices.
- Save and test the code regularly to prevent data loss.

7. Conclusion / Discussion

This experiment demonstrates **dynamic data management using linked lists**.

Students learned to:

- Implement **traversal** to access all nodes.
- Perform **searching** to find elements and positions.
- Insert nodes at the **beginning, end, or middle** of the list.
- Delete nodes safely and manage dynamic memory.
- Compare **linked lists vs arrays** for insertion and deletion efficiency.

Time complexity of operations:

- Traversal: $O(n)$
- Searching: $O(n)$
- Insertion at beginning: $O(1)$, at middle/end: $O(n)$
- Deletion at beginning: $O(1)$, at middle/end: $O(n)$

By completing this experiment, students gain **hands-on experience with pointer-based dynamic structures**, a critical foundation for stacks, queues, and more advanced data structures.

Lab 8

Experiment [CO2:PO3]

Name of the Experiment

Experiment on Linked List: Traversing, Searching, Insertion, and Deletion in a Doubly Linked List and Circular Linked List

1. Introduction / Description

A **doubly linked list (DLL)** is a dynamic linear data structure where each node contains:

- Data
- Pointer to the **next node**
- Pointer to the **previous node**

A **circular linked list (CLL)** is a variation where the **last node points back to the first node**, forming a circular structure.

In this experiment, students will perform the following operations:

- **Traversal** – Display all elements sequentially in forward and backward directions (for DLL) or forward direction (for CLL).
- **Searching** – Find a node containing a specific value.
- **Insertion** – Add a node at the beginning, end, or a specific position.
- **Deletion** – Remove a node from the beginning, end, or a specific position.

These operations are essential to understand **dynamic memory management, pointer manipulation, and optimized data structures**.

2. Required Equipment / Software

Hardware:

- Computer with standard configuration
- Keyboard and mouse

Software:

- C Compiler (Code::Blocks / Turbo C / GCC / Dev-C++)
- Text editor (VS Code / Sublime Text / Notepad++)
- Operating System: Windows or Linux

3. Procedure

Step 1: Setup Environment

- Open the C programming IDE (e.g., Code::Blocks or Turbo C++).
- Create a new project named Doubly_Circular_LinkedList.
- Write or copy the provided program code into the editor.

Step 2: Compile and Run

- Compile the program and resolve any syntax errors.
- Execute the program and interactively perform each operation.

Step 3: Steps to Perform

1. Choose operations from the menu for DLL or CLL: Traversal, Searching, Insertion, Deletion.
2. Input node values and positions as required.
3. Observe the changes in the linked list after each operation.

4. Algorithm / Pseudocode

A. Traversal

- DLL: Start from head, traverse forward using next pointers; traverse backward using prev pointers.
- CLL: Start from head, move forward using next until returning to head.

B. Searching

1. Start from head.
2. While current node is not NULL (or until loop returns to head in CLL):
 - Compare node data with target.
 - If found, display position; else continue.

C. Insertion

1. Create a new node with input data.
2. Depending on position:
 - Beginning: Update head pointers accordingly.
 - End: Traverse to last node, update next and prev pointers.
 - Specific position: Traverse, adjust pointers to insert.

D. Deletion

1. Depending on position:
 - Beginning: Update head, adjust pointers, free node.
 - End: Traverse to last node, adjust previous node's pointer, free node.
 - Specific position: Traverse to node, update adjacent nodes, free memory.

5. C Program Implementation (Doubly Linked List Example)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* next;
```

```
    struct Node* prev;
```

```
};
```

```
struct Node* head = NULL;
```

```
// Traversal
```

```
void traverse() {
```

```
    struct Node* ptr = head;
```

```
    if (ptr == NULL) {
```

```
        printf("List is empty.\n");
```

```
        return;
```

```
    }
```

```
    printf("Forward: ");
```

```

while (ptr != NULL) {
    printf("%d ", ptr->data);
    if (ptr->next == NULL) break;
    ptr = ptr->next;
}
printf("\nBackward: ");
while (ptr != NULL) {
    printf("%d ", ptr->data);
    ptr = ptr->prev;
}
printf("\n");
}

// Search
void search(int key) {
    struct Node* ptr = head;
    int pos = 0, found = 0;
    while (ptr != NULL) {
        if (ptr->data == key) {
            printf("Element %d found at position %d\n", key, pos);
            found = 1;
            break;
        }
        ptr = ptr->next;
        pos++;
    }
}

```

```

    }

    if (!found) printf("Element %d not found.\n", key);
}

// Insertion at end

void insertEnd(int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = NULL;

    if (head == NULL) {
        newNode->prev = NULL;
        head = newNode;
        return;
    }

    struct Node* ptr = head;
    while (ptr->next != NULL)
        ptr = ptr->next;

    ptr->next = newNode;
    newNode->prev = ptr;
}

// Deletion from end

```

```

void deleteEnd() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* ptr = head;
    if (ptr->next == NULL) {
        free(head);
        head = NULL;
        return;
    }

    while (ptr->next != NULL)
        ptr = ptr->next;

    ptr->prev->next = NULL;
    free(ptr);
}

int main() {
    int choice, val, key;
    while (1) {
        printf("\n--- Doubly Linked List Operations ---\n");
        printf("1. Traversal\n2. Search\n3. Insert at End\n4. Delete from End\n5. Exit\n");
    }
}

```

```

printf("Enter your choice: ");

scanf("%d", &choice);


switch (choice) {

    case 1: traverse(); break;

    case 2:

        printf("Enter element to search: ");

        scanf("%d", &key);

        search(key);

        break;

    case 3:

        printf("Enter value to insert: ");

        scanf("%d", &val);

        insertEnd(val);

        break;

    case 4: deleteEnd(); break;

    case 5: return 0;

    default: printf("Invalid choice! Try again.\n");

}

}

}

```

A similar approach can be extended for **circular linked lists**, with the last node pointing to head and traversal terminating upon returning to head.

6. Precautions

- Always check for **NULL pointers** before accessing nodes.
- Validate positions for insertion and deletion.
- Free memory for deleted nodes to avoid **memory leaks**.
- Avoid negative indices or invalid positions.
- Test thoroughly before switching between DLL and CLL operations.

7. Conclusion / Discussion

This experiment demonstrates the implementation and manipulation of **doubly and circular linked lists**.

Students learned to:

- Traverse lists in forward and backward directions (DLL) and circular traversal (CLL).
- Search for elements dynamically.
- Insert nodes at the end, beginning, or specific positions.
- Delete nodes safely while managing dynamic memory.
- Compare DLL and CLL efficiency with singly linked lists for specific operations.

Time complexity:

- Traversal: $O(n)$
- Search: $O(n)$
- Insertion at end/beginning: $O(1)$
- Deletion at end/beginning: $O(1)$

By completing this experiment, students gain **hands-on experience with advanced pointer-based dynamic structures**, which is foundational for stacks, queues, and complex data structures like graphs.

Lab 9

Experiment [CO3:PO4]

Name of the Experiment

Experiment on Sorting: Implementing and Analyzing Bubble Sort, Insertion Sort, and Selection Sort Algorithms

1. Introduction / Description

Sorting is a fundamental operation in computer science and algorithms that organizes data in a particular order (ascending or descending). Efficient sorting improves performance in searching, data processing, and management.

In this experiment, students will implement and analyze three common sorting algorithms:

- **Bubble Sort** – Repeatedly compares adjacent elements and swaps them if they are in the wrong order.
- **Insertion Sort** – Builds a sorted portion of the array one element at a time by inserting each new element into its correct position.
- **Selection Sort** – Selects the minimum element from the unsorted portion and swaps it with the first unsorted element.

The experiment emphasizes understanding **algorithm efficiency, time complexity, and array manipulation techniques**.

2. Required Equipment / Software

Hardware:

- Computer with standard configuration
- Keyboard and mouse

Software:

- C Compiler (Code::Blocks / Turbo C / GCC / Dev-C++)
- Text editor (VS Code / Sublime Text / Notepad++)
- Operating System: Windows or Linux

3. Procedure

Step 1: Setup Environment

- Open the C programming IDE (e.g., Code::Blocks or Turbo C++).
- Create a new project named Sorting_Algorithms.
- Write or copy the provided program code into the editor.

Step 2: Compile and Run

- Compile the program and fix any syntax errors.
- Execute the program and test each sorting algorithm.

Step 3: Steps to Perform

1. Input the number of elements and the array elements.
2. Choose an algorithm from the menu: Bubble Sort, Insertion Sort, or Selection Sort.
3. Observe the array after sorting.
4. Record the number of comparisons and swaps (for complexity analysis).

4. Algorithm / Pseudocode

A. Bubble Sort Algorithm

1. Start
2. Repeat for $i = 0$ to $n-1$:
 - Compare adjacent elements $arr[j]$ and $arr[j+1]$
 - Swap if $arr[j] > arr[j+1]$
3. Repeat until no swaps are needed
4. Stop

Time Complexity: Best: $O(n)$, Worst: $O(n^2)$

B. Insertion Sort Algorithm

1. Start
2. For $i = 1$ to $n-1$:
 - Store $arr[i]$ in key
 - Compare key with elements before it
 - Shift larger elements to the right
 - Insert key in the correct position
3. Stop

Time Complexity: Best: $O(n)$, Worst: $O(n^2)$

C. Selection Sort Algorithm

1. Start
2. For $i = 0$ to $n-1$:
 - Find minimum element in unsorted portion
 - Swap it with $arr[i]$
3. Stop

Time Complexity: $O(n^2)$ for all cases

5. C Program Implementation

```
#include <stdio.h>
```

```
void bubbleSort(int arr[], int n) {
    int i, j, temp, swapCount=0;
    for(i=0; i<n-1; i++) {
        for(j=0; j<n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
                swapCount++;
            }
        }
    }
    printf("Bubble Sort completed with %d swaps.\n", swapCount);
}
```

```
void insertionSort(int arr[], int n) {
    int i, j, key, shiftCount=0;
```

```

for(i=1; i<n; i++) {
    key = arr[i];
    j = i-1;
    while(j>=0 && arr[j]>key) {
        arr[j+1] = arr[j];
        j--;
        shiftCount++;
    }
    arr[j+1] = key;
}

printf("Insertion Sort completed with %d shifts.\n", shiftCount);
}

```

```

void selectionSort(int arr[], int n) {
    int i, j, minIdx, temp, swapCount=0;
    for(i=0; i<n-1; i++) {
        minIdx = i;
        for(j=i+1; j<n; j++)
            if(arr[j]<arr[minIdx])
                minIdx = j;
        if(minIdx != i) {
            temp = arr[i];
            arr[i] = arr[minIdx];
            arr[minIdx] = temp;
            swapCount++;
        }
    }
}

```

```

    }

}

printf("Selection Sort completed with %d swaps.\n", swapCount);
}

void printArray(int arr[], int n) {
    int i;
    for(i=0; i<n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[100], n, choice, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for(i=0; i<n; i++)
        scanf("%d", &arr[i]);

    while(1) {
        printf("\n--- Sorting Algorithms ---\n");
        printf("1. Bubble Sort\n2. Insertion Sort\n3. Selection Sort\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
    }
}

```

```
int arrCopy[100];

for(i=0;i<n;i++) arrCopy[i]=arr[i];

switch(choice) {

    case 1:

        bubbleSort(arrCopy, n);

        printArray(arrCopy, n);

        break;

    case 2:

        insertionSort(arrCopy, n);

        printArray(arrCopy, n);

        break;

    case 3:

        selectionSort(arrCopy, n);

        printArray(arrCopy, n);

        break;

    case 4:

        return 0;

    default:

        printf("Invalid choice! Try again.\n");

}

}

}
```

6. Precautions

- Ensure array indices are valid (0 to n-1).
- Avoid exceeding the maximum array size.
- Verify input data to prevent incorrect results.
- Test each sorting algorithm separately to understand its behavior.
- Keep a backup of original array for comparative analysis.

7. Conclusion / Discussion

This experiment demonstrates **sorting operations and their time complexity**.

Students learned to:

- Implement Bubble, Insertion, and Selection sort algorithms.
- Analyze the number of comparisons, swaps, and shifts.
- Compare algorithm efficiency and understand the best, average, and worst-case scenarios.
- Understand how sorting is foundational for searching algorithms and higher-level data structure operations.

Time Complexity Overview:

- **Bubble Sort:** Best: $O(n)$, Worst: $O(n^2)$
- **Insertion Sort:** Best: $O(n)$, Worst: $O(n^2)$
- **Selection Sort:** Best/Worst: $O(n^2)$

By completing this experiment, students gain **practical experience in algorithm implementation and performance analysis**.

Lab 10

Experiment [CO3:PO4]

Name of the Experiment

Experiment on Sorting: Implementing and Analyzing Merge Sort and Quick Sort Algorithms

1. Introduction / Description

Sorting is a fundamental operation in data structures and algorithms that organizes data in a specific order (ascending or descending). Efficient sorting improves performance in searching, data processing, and large-scale data management.

In this experiment, students will implement and analyze two efficient sorting algorithms:

- **Merge Sort** – A divide-and-conquer algorithm that divides the array into halves, recursively sorts each half, and merges the sorted halves.
- **Quick Sort** – A divide-and-conquer algorithm that selects a pivot element, partitions the array into elements less than and greater than the pivot, and recursively sorts the partitions.

The experiment emphasizes understanding **algorithm efficiency, recursion, and divide-and-conquer techniques**.

2. Required Equipment / Software

Hardware:

- Computer with standard configuration
- Keyboard and mouse

Software:

- C Compiler (Code::Blocks / Turbo C / GCC / Dev-C++)
- Text editor (VS Code / Sublime Text / Notepad++)
- Operating System: Windows or Linux

3. Procedure

Step 1: Setup Environment

- Open the C programming IDE (e.g., Code::Blocks or Turbo C++).
- Create a new project named Merge_Quick_Sort.
- Write or copy the provided program code into the editor.

Step 2: Compile and Run

- Compile the program and fix any syntax errors.
- Execute the program and test each sorting algorithm.

Step 3: Steps to Perform

1. Input the number of elements and the array elements.
2. Choose an algorithm from the menu: Merge Sort or Quick Sort.
3. Observe the array after sorting.
4. Record the number of recursive calls and element comparisons (for complexity analysis).

4. Algorithm / Pseudocode

A. Merge Sort Algorithm

1. Start
2. Divide the array into two halves
3. Recursively sort each half
4. Merge the sorted halves
5. Stop

Time Complexity: Best, Average, Worst: $O(n \log n)$

B. Quick Sort Algorithm

1. Start
2. Select a pivot element
3. Partition the array such that elements $<$ pivot go left, $>$ pivot go right
4. Recursively apply Quick Sort on left and right partitions
5. Stop

Time Complexity:

- Best/Average: $O(n \log n)$
- Worst: $O(n^2)$ (occurs when pivot selection is poor)

5. C Program Implementation

```
#include <stdio.h>

void merge(int arr[], int l, int m, int r) {
    int i, j, k;

    int n1 = m - l + 1;

    int n2 = r - m;

    int L[n1], R[n2];

    for(i = 0; i < n1; i++) L[i] = arr[l + i];

    for(j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    i = 0; j = 0; k = l;

    while(i < n1 && j < n2) {
        if(L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }

    while(i < n1) arr[k++] = L[i++];

    while(j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if(l < r) {
        int m = l + (r - l)/2;

        mergeSort(arr, l, m);

        mergeSort(arr, m+1, r);

        merge(arr, l, m, r);
    }
}
```

```

    }
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1, temp;
    for(int j = low; j < high; j++) {
        if(arr[j] <= pivot) {
            i++;
            temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
        }
    }
    temp = arr[i+1]; arr[i+1] = arr[high]; arr[high] = temp;
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if(low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi-1);
        quickSort(arr, pi+1, high);
    }
}

void printArray(int arr[], int n) {
    for(int i=0; i<n; i++) printf("%d ", arr[i]);
    printf("\n");
}

```

```

int main() {

    int arr[100], n, choice, i;

    printf("Enter number of elements: ");

    scanf("%d", &n);

    printf("Enter elements: ");

    for(i=0; i<n; i++) scanf("%d", &arr[i]);


    while(1) {

        int arrCopy[100];

        for(i=0; i<n; i++) arrCopy[i]=arr[i];

        printf("\n--- Sorting Algorithms ---\n");

        printf("1. Merge Sort\n2. Quick Sort\n3. Exit\n");

        printf("Enter your choice: ");

        scanf("%d", &choice);

        switch(choice) {

            case 1:

                mergeSort(arrCopy, 0, n-1);

                printf("Array after Merge Sort: ");

                printArray(arrCopy, n);

                break;

            case 2:

                quickSort(arrCopy, 0, n-1);

                printf("Array after Quick Sort: ");

                printArray(arrCopy, n);

                break;

```

```

case 3:

    return 0;

default:

    printf("Invalid choice! Try again.\n");

}

}

}

```

6. Precautions

- Ensure array indices are valid (0 to n-1).
- Avoid exceeding maximum array size.
- Ensure sufficient stack memory for recursion.
- Test with small arrays first to verify correct sorting.
- Save code frequently to prevent data loss.

7. Conclusion / Discussion

This experiment demonstrates **efficient sorting algorithms using recursion and divide-and-conquer techniques**.

Students learned to:

- Implement Merge Sort and Quick Sort.
- Analyze recursive behavior and algorithm efficiency.
- Compare time complexity:
 - **Merge Sort:** $O(n \log n)$ (all cases)
 - **Quick Sort:** $O(n \log n)$ (average), $O(n^2)$ (worst)
- Understand practical advantages of efficient sorting for large datasets.

By completing this experiment, students gain practical knowledge of **advanced sorting algorithms** and their significance in algorithm design and data structure optimization.

Lab 11

Experiment [CO4:PO5]

Name of the Experiment

Experiment on Tree: Traversing Binary Tree, Binary Search Tree, Insertion and Deletion on Binary Search Tree

1. Introduction / Description

Trees are hierarchical data structures widely used in computer science to represent relationships between data elements. A **Binary Tree (BT)** is a tree where each node has at most two children, typically referred to as the left and right child. A **Binary Search Tree (BST)** is a special kind of binary tree in which the left subtree contains nodes with values smaller than the root and the right subtree contains nodes with values greater than the root.

In this experiment, students will perform the following operations:

- **Tree Traversals** – Visit nodes of a binary tree in **Inorder**, **Preorder**, and **Postorder** sequences.
- **Binary Search Tree Operations** – Insert new nodes, delete existing nodes, and traverse the tree.
- Understanding these operations is crucial for efficient **data storage, retrieval, and hierarchical organization** in computer science applications.

2. Required Equipment / Software

Hardware:

- Computer with standard configuration
- Keyboard and mouse

Software:

- C Compiler (Code::Blocks / Turbo C / GCC / Dev-C++)
- Text editor (VS Code / Sublime Text / Notepad++)
- Operating System: Windows or Linux

3. Procedure

Step 1: Setup Environment

- Open the C programming IDE (e.g., Code::Blocks or Turbo C++).
- Create a new project named BinaryTree_BST.
- Write or copy the provided program code into the editor.

Step 2: Compile and Run

- Compile the program and fix any syntax errors.
- Execute the program and test each operation.

Step 3: Steps to Perform

1. Input elements to create a Binary Search Tree.
2. Perform traversals (Inorder, Preorder, Postorder) to verify the tree structure.
3. Insert new nodes and observe the BST property.
4. Delete nodes and verify tree integrity through traversals.
5. Record output and observe how BST operations maintain sorted order.

4. Algorithm / Pseudocode

A. Tree Traversals

1. **Inorder Traversal (Left, Root, Right)**
 - Visit left subtree
 - Visit root
 - Visit right subtree
2. **Preorder Traversal (Root, Left, Right)**
 - Visit root
 - Visit left subtree
 - Visit right subtree
3. **Postorder Traversal (Left, Right, Root)**
 - Visit left subtree
 - Visit right subtree
 - Visit root

B. BST Insertion Algorithm

1. Start
2. If the tree is empty, create a new node as root
3. Compare new value with current node:
 - If smaller, move to left child
 - If larger, move to right child
4. Repeat recursively until correct position is found
5. Insert new node
6. Stop

C. BST Deletion Algorithm

1. Start
2. Search the node to delete
3. If node has no children, delete directly
4. If node has one child, replace node with child
5. If node has two children, find inorder successor or predecessor, replace, and delete successor/predecessor
6. Stop

5. C Program Implementation

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
```

```
    int data;
```

```
    struct Node *left, *right;
```

```
};
```

```
// Create new node
```

```
struct Node* createNode(int val) {
```

```
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
```

```
    node->data = val;
```

```
    node->left = node->right = NULL;
```

```
    return node;
```

```
}
```

```
// Inorder Traversal
```

```
void inorder(struct Node* root) {
```

```
    if(root != NULL) {
```

```
        inorder(root->left);
```

```

    printf("%d ", root->data);

    inorder(root->right);

}

}

```

// Preorder Traversal

```

void preorder(struct Node* root) {

    if(root != NULL) {

        printf("%d ", root->data);

        preorder(root->left);

        preorder(root->right);

    }

}

```

// Postorder Traversal

```

void postorder(struct Node* root) {

    if(root != NULL) {

        postorder(root->left);

        postorder(root->right);

        printf("%d ", root->data);

    }

}

```

// Insert into BST

```

struct Node* insertBST(struct Node* root, int val) {

    if(root == NULL) return createNode(val);

    if(val < root->data)

        root->left = insertBST(root->left, val);

```

```

else if(val > root->data)

    root->right = insertBST(root->right, val);

return root;

}

// Find minimum node

struct Node* minValueNode(struct Node* node) {

    struct Node* current = node;

    while(current && current->left != NULL)

        current = current->left;

    return current;

}

// Delete node in BST

struct Node* deleteBST(struct Node* root, int val) {

    if(root == NULL) return root;

    if(val < root->data)

        root->left = deleteBST(root->left, val);

    else if(val > root->data)

        root->right = deleteBST(root->right, val);

    else {

        if(root->left == NULL) {

            struct Node* temp = root->right;

            free(root);

            return temp;

        } else if(root->right == NULL) {

            struct Node* temp = root->left;

```

```

    free(root);

    return temp;
}

struct Node* temp = minValueNode(root->right);

root->data = temp->data;

root->right = deleteBST(root->right, temp->data);
}

return root;
}

int main() {

    struct Node* root = NULL;

    int choice, val;

    while(1) {

        printf("\n--- BST Operations ---\n");

        printf("1. Insert\n2. Delete\n3. Inorder Traversal\n4. Preorder Traversal\n5. Postorder Traversal\n6. Exit\n");

        printf("Enter your choice: ");

        scanf("%d", &choice);

        switch(choice) {

            case 1:

                printf("Enter value to insert: ");

                scanf("%d", &val);

                root = insertBST(root, val);

                break;

            case 2:

                printf("Enter value to delete: ");

```

```
scanf("%d", &val);

root = deleteBST(root, val);

break;

case 3:

    printf("Inorder: ");

    inorder(root);

    printf("\n");

    break;

case 4:

    printf("Preorder: ");

    preorder(root);

    printf("\n");

    break;

case 5:

    printf("Postorder: ");

    postorder(root);

    printf("\n");

    break;

case 6:

    return 0;

default:

    printf("Invalid choice! Try again.\n");

}

}

}
```

6. Precautions

- Ensure correct memory allocation and deallocation to avoid memory leaks.
- Avoid duplicate values when inserting in BST (if required).
- Validate node existence before deletion.
- Test with small trees first to verify correctness of traversals.
- Save code frequently to prevent data loss.

7. Conclusion / Discussion

This experiment demonstrates the **implementation of Binary Trees and Binary Search Trees** in C.

Students learned to:

- Traverse binary trees using Inorder, Preorder, and Postorder methods.
- Insert and delete nodes while maintaining the BST property.
- Understand recursive operations and tree-based hierarchical data representation.
- Analyze **time complexity**:
 - Traversals: $O(n)$
 - Insertion/Deletion in BST: $O(h)$ where h is the height of the tree ($O(\log n)$ for balanced BST, $O(n)$ for skewed BST)

By completing this experiment, students gain practical understanding of tree data structures and their importance in **efficient data storage and retrieval**.

Lab 12

Experiment [CO4:PO5]

Name of the Experiment

Experiment on Insertion and Deletion on Heap, Heap Sort, Hash Table

1. Introduction / Description

Heaps and hash tables are fundamental data structures that provide efficient ways of organizing and accessing data.

- **Heap:** A complete binary tree where each parent node satisfies the heap property:
 - **Max-Heap:** Parent $\geq$ Children
 - **Min-Heap:** Parent $\leq$ Children

Heap operations allow for efficient priority management, insertion, deletion, and sorting (Heap Sort).

- **Heap Sort:** A comparison-based sorting algorithm that uses a binary heap to sort elements efficiently.
- **Hash Table:** A data structure that maps keys to values for efficient lookup, insertion, and deletion using a hash function and handling collisions.

In this experiment, students will:

- Perform insertion and deletion in a heap.
 - Implement Heap Sort to sort elements.
 - Create a hash table, insert, search, and delete elements.
- These operations illustrate **efficient data access, memory management, and algorithmic problem-solving**.

2. Required Equipment / Software

Hardware:

- Computer with standard configuration
- Keyboard and mouse

Software:

- C Compiler (Code::Blocks / Turbo C / GCC / Dev-C++)
- Text editor (VS Code / Sublime Text / Notepad++)

- Operating System: Windows or Linux

3. Procedure

Step 1: Setup Environment

- Open the C programming IDE (e.g., Code::Blocks or Turbo C++).
- Create a new project named Heap_Hash_Operations.
- Write or copy the provided program code into the editor.

Step 2: Compile and Run

- Compile the program and fix any syntax errors.
- Execute the program and test each operation.

Step 3: Steps to Perform

1. Input elements into the heap.
2. Perform insertion and deletion operations and observe the heap property.
3. Perform Heap Sort on the heap elements and observe the sorted output.
4. Create a hash table, insert elements, search, and delete keys.
5. Record output and verify correctness.

4. Algorithm / Pseudocode

A. Heap Insertion

1. Start
2. Insert element at the next available position (bottom-most, rightmost)
3. Compare with parent and swap if heap property is violated
4. Repeat until heap property is restored
5. Stop

B. Heap Deletion

1. Start
2. Remove root element
3. Replace root with the last element
4. Heapify the root to restore heap property
5. Stop

C. Heap Sort

1. Build max-heap from the array
2. Swap the root with the last element
3. Reduce heap size by 1 and heapify the root

4. Repeat until heap size = 1
5. Stop

D. Hash Table Operations

1. **Insertion:** Compute hash index and insert key; handle collisions using linear probing.
2. **Search:** Compute hash index and check key; probe until key is found or table end.
3. **Deletion:** Search key and mark as deleted.

5. C Program Implementation

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 100

// Heap Implementation

int heap[MAX];

int heap_size = 0;

void swap(int *a,int *b){ int temp=*a; *a=*b; *b=temp; }

void heapify_up(int i){
    while(i != 0 && heap[(i-1)/2] < heap[i]){
        swap(&heap[i], &heap[(i-1)/2]);
        i = (i-1)/2;
    }
}

void heapify_down(int i){
    int left = 2*i + 1;
    int right = 2*i + 2;
    int largest = i;

    if(left < heap_size && heap[left] > heap[largest]) largest = left;
```

```

    if(right < heap_size && heap[right] > heap[largest]) largest = right;

    if(largest != i){
        swap(&heap[i], &heap[largest]);

        heapify_down(largest);
    }
}

void insert_heap(int val){
    heap[heap_size] = val;

    heap_size++;

    heapify_up(heap_size-1);
}

void delete_heap(){
    if(heap_size <= 0){ printf("Heap empty!\n"); return;}

    heap[0] = heap[heap_size-1];

    heap_size--;

    heapify_down(0);
}

void print_heap(){
    for(int i=0;i<heap_size;i++) printf("%d ",heap[i]);

    printf("\n");
}

```

// Heap Sort

```

void heap_sort(int arr[], int n){
    for(int i=n/2-1;i>=0;i--) heapify_down(i);
}

```

```

for(int i=n-1;i>0;i--){
    swap(&heap[0], &heap[i]);
    heap_size--;
    heapify_down(0);
}
}

// Hash Table Implementation

#define TABLE_SIZE 10

int hash_table[TABLE_SIZE];

int hash(int key){ return key % TABLE_SIZE; }

void insert_hash(int key){
    int index = hash(key);
    int original = index;
    while(hash_table[index] != -1){
        index = (index+1) % TABLE_SIZE;
        if(index == original){ printf("Table full!\n"); return;}
    }
    hash_table[index] = key;
}

int search_hash(int key){
    int index = hash(key);
    int original = index;
    while(hash_table[index] != -1){
        if(hash_table[index] == key) return index;
        index = (index+1) % TABLE_SIZE;
    }
}

```

```

        if(index == original) break;
    }

    return -1;
}

void delete_hash(int key){
    int idx = search_hash(key);

    if(idx != -1) hash_table[idx] = -1;
}

int main(){
    int choice, val;

    for(int i=0;i<TABLE_SIZE;i++) hash_table[i] = -1;

    while(1){
        printf("\n--- Heap & Hash Table Operations ---\n");

        printf("1. Insert Heap\n2. Delete Heap\n3. Print Heap\n4. Insert Hash\n5. Search Hash\n6. Delete Hash\n7. Exit\n");

        printf("Enter your choice: "); scanf("%d",&choice);

        switch(choice){
            case 1: printf("Enter value to insert in heap: "); scanf("%d",&val); insert_heap(val); break;
            case 2: delete_heap(); break;
            case 3: printf("Heap elements: "); print_heap(); break;
            case 4: printf("Enter value to insert in hash table: "); scanf("%d",&val); insert_hash(val); break;
            case 5: printf("Enter value to search in hash table: "); scanf("%d",&val);

                    int idx = search_hash(val);

                    if(idx != -1) printf("Found at index %d\n",idx); else printf("Not found\n"); break;
        }
    }
}

```

```

        case 6: printf("Enter value to delete from hash table: "); scanf("%d",&val);
delete_hash(val); break;

        case 7: return 0;

        default: printf("Invalid choice!\n");

    }

}

}

```

6. Precautions

- Ensure the heap is not empty before deletion.
- Maintain heap property after every insertion and deletion.
- Initialize hash table with empty values (-1) and handle collisions properly.
- Avoid inserting duplicate keys if not required.
- Validate array sizes and indices to prevent overflow.
- Test with small datasets first to verify correctness.

7. Conclusion / Discussion

This experiment demonstrates **advanced tree and table data structures**:

Students learned to:

- Insert and delete elements in a heap while maintaining heap property.
- Implement Heap Sort for efficient sorting of elements.
- Create and manipulate a hash table with basic collision handling.
- Analyze **time complexity**:
 - Heap insertion/deletion: $O(\log n)$
 - Heap Sort: $O(n \log n)$
 - Hash table insertion/search/deletion: $O(1)$ average, $O(n)$ worst-case

By completing this experiment, students gain practical skills in implementing **efficient data structures for priority management, sorting, and fast data retrieval**.

Lab 13

Lab Test Instructions – Data Structures

1. General Guidelines

1. Arrive **punctually**; late entry may result in reduced marks or disqualification.
2. Bring your **lab notebook, pen, and student ID**.
3. Electronic devices (mobile phones, smart watches) are **not allowed**, except for approved lab equipment.
4. Ensure the **IDE or compiler** is properly installed and ready on your system before the test.
5. Read **all instructions carefully** before starting the test.

2. Test Duration

- Total Time: **1.5 – 2 hours**
- Marks: **20 – 30** (as per course rubric)

3. Test Structure

The lab test may include **any combination** of the following:

1. **Programming Problems**
 - Implement a data structure (Array, Stack, Queue, Linked List, Tree, Heap, Graph, etc.)
 - Perform operations such as insertion, deletion, traversal, search, or sorting
2. **Algorithm Analysis**
 - Write pseudocode or flowchart
 - Analyze **time and space complexity**
3. **Output Verification**
 - Execute your code and display results
 - Demonstrate correct functionality for **sample inputs**

4. Procedure

1. Read the question carefully and understand the requirements.
2. Plan the **algorithm or steps** on paper before coding.
3. Write clean and **well-commented code** in C, C++, or Java (as instructed).
4. Compile and execute your code; ensure it runs without errors.

5. Record **outputs and observations** in your lab notebook or printout if required.
6. Submit **source code, output screenshots, and lab notebook** (if applicable) to the instructor.

5. Coding Guidelines

- Use **meaningful variable names** and indentation.
- Add **comments** to explain logic for each function or block.
- Avoid using **hardcoded values**; take input from the user.
- Ensure your program handles **boundary cases** (empty array, full stack, etc.).

6. Evaluation Criteria

Marks will be awarded based on:

1. **Correctness of Implementation** – Code runs correctly and meets problem requirements.
2. **Understanding & Application** – Ability to explain the logic and algorithm used.
3. **Output Accuracy** – Output matches expected results for sample inputs.
4. **Code Quality** – Proper formatting, comments, and readability.
5. **Time Management & Discipline** – Completing tasks within the given time frame and following instructions.

7. Precautions

- Avoid copying code from others; plagiarism will result in **zero marks**.
- Save your code frequently to prevent data loss.
- Ensure proper **backup** of your source files.
- Report any **technical issues** to the instructor immediately.

Quiz Instructions – Data Structure

Total Marks: 20

Duration: 20–25 minutes

1. General Guidelines

1. Be **punctual**; late submission may result in zero marks.
2. Bring **pen, paper, and student ID**.

3. **Electronic devices** (mobile phones, smart watches, tablets) are **not allowed** unless explicitly permitted.
4. Read **all questions carefully** before answering.

2. Quiz Format

The quiz may include:

- **Multiple Choice Questions (MCQs)** – Test basic concepts and definitions.
- **Short Answer Questions** – Explain terms, algorithms, or data structure properties.
- **Problem-Solving / Coding Questions** – Small code snippets or algorithm steps (pseudocode).

3. Instructions During Quiz

1. Write **clearly and legibly**.
2. For coding questions, provide **pseudocode or C/C++ code** (as instructed).
3. **Attempt all questions**; partial credit may be given for incomplete solutions.
4. Check your answers for **accuracy and completeness**.
5. Hand over your answer sheet to the instructor at the end of the quiz.

4. Marking Scheme (Example for 20 Marks)

- MCQs: 5 marks
- Short Answer Questions: 5 marks
- Coding / Problem Solving: 10 marks

5. Precautions

- Do not **share answers** with others.
- Ensure all calculations and logic are clearly written.
- Plagiarism or cheating will lead to **zero marks**.